SOFTWARE DESIGN & ARCHITECTURE

Individual Assignment — From Real-World Concepts to Software Objects

Selected Domain	University Course Registration System
Student Name	ROHAIL ABBAS
Semester / Section	3RD SEMESTER
Submission Date	27/10/26

1. Introduction

A University Course Registration System helps students select and register for course sections offered in a semester. It checks prerequisites and seat availability, stores enrollment records, and supports the updating of academic transcripts after final grades are issued. The purpose of this report is to identify the main conceptual classes in this domain and show how selected classes can be represented in a software design.

2. Candidate Conceptual Classes

The following classes were identified by examining the important nouns and concepts in the course registration process. Each class represents a meaningful person, record, rule, place, or academic concept.

No.	Candidate Class	Justification
1	**Student**	Represents a person who registers for courses and receives grades.
2	**Course**	Describes an academic course, including its code, title, and credit hours.
3	**CourseSection**	Represents a particular offering of a course in a specific semester.
4	**Semester**	Identifies the academic time period in which courses are offered.
5	**Enrollment**	Records a student's registration in a course section.
6	**Prerequisite**	Represents a rule specifying prior study required for a course.
7	**Transcript**	Stores a student's academic course and grade history.
8	**Instructor**	Represents a teacher assigned to a course section.
9	**Classroom**	Represents the room where a course section may be taught.
10	**Schedule**	Represents the days and times when a course section meets.

3. Attributes and Methods of Three Classes

Attributes describe an object's state, while methods describe the actions or responsibilities it supports.

Class	Attributes (State)	Methods (Behavior)
Student	**Attributes:** studentID : String; name : String; email : String; registeredSections : List; cumulativeGPA : Decimal.	**Methods:** register(section); drop(section); hasCompleted(course) : Boolean; getTranscript() : Transcript; updateGPA().
CourseSection	**Attributes:** sectionID : String; course : Course; semester : Semester; instructor : Instructor; capacity : Integer; currentEnrollment : Integer; schedule : Schedule; room : Classroom.	**Methods:** isFull() : Boolean; addStudent(student); removeStudent(student); hasSeatAvailable() : Boolean; getRemainingSeats() : Integer.
Enrollment	**Attributes:** enrollmentID : String; student : Student; courseSection : CourseSection; enrollmentDate : Date; status : String; finalGrade : String.	**Methods:** create(); cancel(); updateStatus(status); recordFinalGrade(grade); getEnrollmentDetails().

4. Abstraction Decisions

Abstraction means keeping the details that matter to the registration system and leaving out details that do not help it perform its main tasks. For the Student class, I include the student ID, name, email, registered sections, and cumulative GPA because these details support registration and academic records. I leave out details such as favorite color, shoe size, hobbies, and music taste because they are not needed to register for courses or update a transcript. I model CourseSection separately from Course because one course can be offered in different semesters, with different instructors, schedules, rooms, and seat limits. I also use Enrollment as a separate class because the system needs a record connecting a particular student to a particular course section. These choices keep the model focused and avoid storing unrelated information.

5. One Invented Software Object

RegistrationController is an invented software object that coordinates the registration use case. It receives a registration request, asks the relevant objects to check prerequisites and seat availability, creates an Enrollment record when the checks succeed, and returns a success or failure result. A controller does not represent a physical object in the university; it is added to the design model to coordinate work between domain objects and keep registration logic organized.

6. Conclusion

The University Course Registration System can be modeled using conceptual classes that represent its main people, academic offerings, rules, and records. Defining attributes and methods clarifies each class's state and responsibilities. Abstraction keeps the model focused on relevant information, while the invented RegistrationController helps coordinate the software workflow. Together, these decisions provide a useful starting point for designing and implementing the system.